

AWS Deployment Guide

Complete enterprise-grade deployment guide for AuthiChain on AWS.

Why AWS?

Pros:

-  **Enterprise-ready** - Used by Fortune 500 companies
-  **Full control** - Customize everything
-  **Scalability** - Handle millions of users
-  **Global reach** - 30+ regions worldwide
-  **Comprehensive services** - Complete ecosystem
-  **Best security** - Compliance certifications

Cons:

-  Complex setup - Steeper learning curve
-  More expensive - Higher baseline costs
-  Manual configuration - Less “magic”

Pricing:

- **Starter Setup:** ~\$50-100/month
- **Production Setup:** ~\$200-500/month
- **Enterprise Setup:** \$1,000+/month

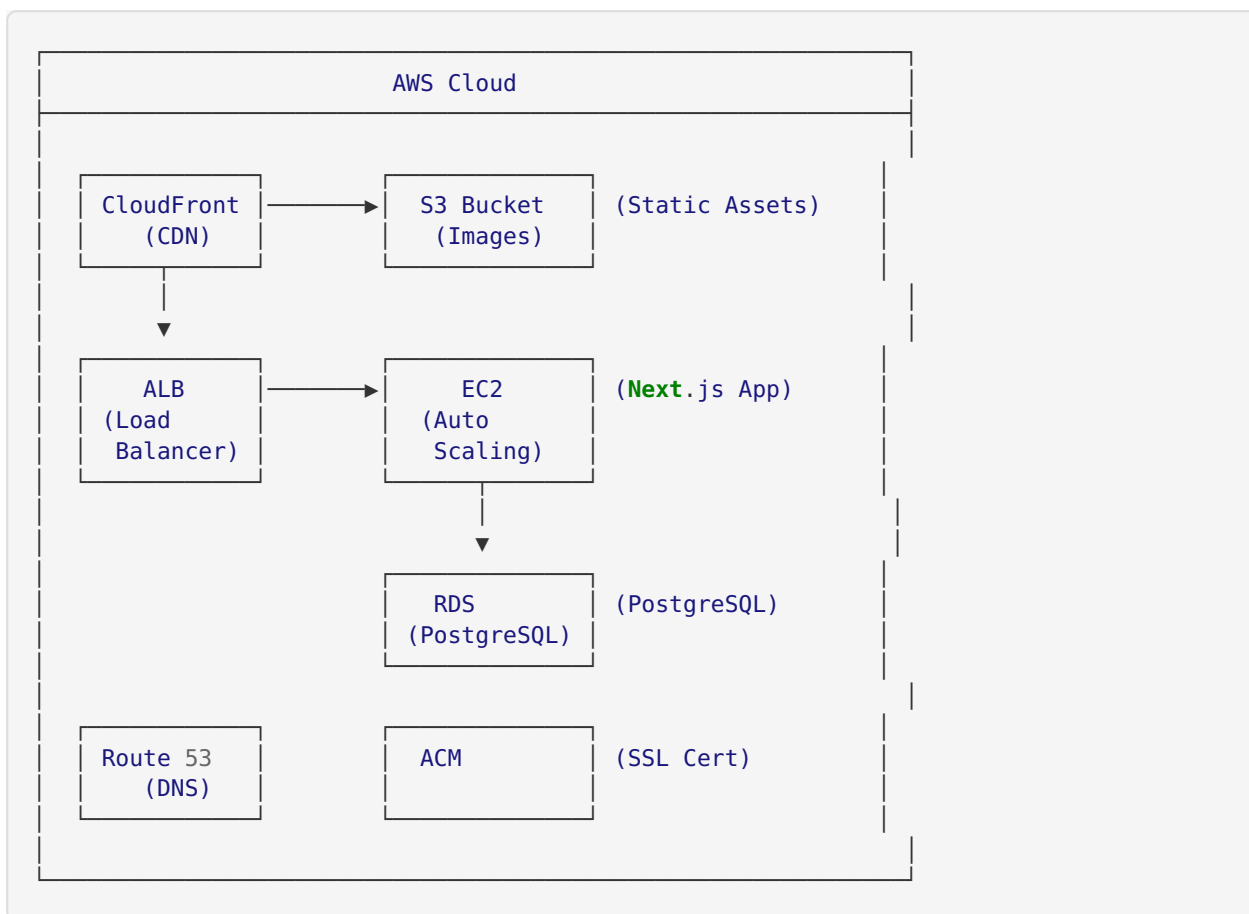
Best For: Large-scale production deployments, enterprise clients, compliance requirements

Prerequisites

-  AWS Account (with billing enabled)
 -  AWS CLI installed
 -  Basic AWS knowledge (EC2, RDS, S3, etc.)
 -  All API keys ready (see [API_KEYS_SETUP.md](#) (./API_KEYS_SETUP.md))
 -  Domain name registered
 -  GitHub account with AuthiChain repo
-

Architecture Overview

We'll deploy using this architecture:



Deployment Steps

Step 1: Install AWS CLI

macOS:

```
brew install awscli
```

Linux:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Windows:

Download from: <https://awscli.amazonaws.com/AWSCLIV2.msi>

Verify Installation:

```
aws --version
```

Step 2: Configure AWS Credentials

1. Create IAM User:

- Go to [AWS IAM Console](https://console.aws.amazon.com/iam/) (https://console.aws.amazon.com/iam/)
- Click "Users" → "Add user"
- Username: `authichain-deployer`
- Access type: ☒ Programmatic access
- Click "Next: Permissions"
- Attach policies:
 - `AmazonEC2FullAccess`
 - `AmazonRDSFullAccess`
 - `AmazonS3FullAccess`
 - `CloudFrontFullAccess`
 - `AWSCertificateManagerFullAccess`
 - Click "Create user"
 - **Save the Access Key ID and Secret Access Key!**

2. Configure CLI:

```
bash
aws configure
```

Enter:

```
AWS Access Key ID: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name: us-east-1
Default output format: json
```

Step 3: Create RDS PostgreSQL Database

1. Launch RDS Instance:

```
bash
aws rds create-db-instance \
  --db-instance-identifier authichain-db \
  --db-instance-class db.t3.micro \
  --engine postgres \
  --engine-version 15.3 \
  --master-username authichain_admin \
  --master-user-password YOUR_STRONG_PASSWORD \
  --allocated-storage 20 \
  --storage-type gp2 \
  --publicly-accessible \
  --backup-retention-period 7 \
  --multi-az false \
  --db-name authichain \
  --tags Key=Name,Value=AuthiChain-Production
```

2. Wait for Database (5-10 minutes):

```
bash
aws rds wait db-instance-available --db-instance-identifier authichain-db
```

3. Get Database Endpoint:

```
bash
aws rds describe-db-instances \
  --db-instance-identifier authichain-db \
  --query 'DBInstances[0].Endpoint.Address' \
  --output text
```

4. Connection String:

```
postgresql://authichain_admin:YOUR_PASSWORD@authichain-db.xxxxxxxxxx.us-east-1.rds.amazonaws.com:5432/authichain
```

Step 4: Configure Security Group for RDS**1. Get RDS Security Group ID:**

```
bash
aws rds describe-db-instances \
  --db-instance-identifier authichain-db \
  --query 'DBInstances[0].VpcSecurityGroups[0].VpcSecurityGroupId' \
  --output text
```

2. Allow PostgreSQL Access:

```
bash
aws ec2 authorize-security-group-ingress \
  --group-id sg-xxxxxxx \
  --protocol tcp \
  --port 5432 \
  --cidr 0.0.0.0/0
```

 **Security Note:** For production, replace `0.0.0.0/0` with specific IP ranges!

Step 5: Create EC2 Instance**1. Create Key Pair:**

```
``bash
aws ec2 create-key-pair \
  -key-name authichain-key \
  -query 'KeyMaterial' \
  -output text > authichain-key.pem

chmod 400 authichain-key.pem
``
```

1. Launch EC2 Instance:

```
bash
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafa1f0 \
  --instance-type t3.medium \
  --key-name authichain-key \
  --security-group-ids sg-xxxxxxx \
```

```
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=AuthiChain-App}]' \
--user-data file://setup-ec2.sh
```

2. Create setup-ec2.sh :

```
```bash
#!/bin/bash
Update system
sudo yum update -y

Install Node.js 18
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install -y nodejs

Install Git
sudo yum install -y git

Install PM2
sudo npm install -g pm2

Clone repository
cd /home/ec2-user
git clone https://github.com/YOUR_USERNAME/authichain.git
cd authichain/app

Install dependencies
npm install

Build application
npm run build

Start with PM2
pm2 start npm --name "authichain" -- start
pm2 save
pm2 startup
```
```

1. Get Instance Public IP:

```
bash
aws ec2 describe-instances \
--filters "Name=tag:Name,Values=AuthiChain-App" \
--query 'Reservations[0].Instances[0].PublicIpAddress' \
--output text
```

Step 6: Configure Environment Variables on EC2

1. SSH into EC2:

```
bash
ssh -i authichain-key.pem ec2-user@YOUR_EC2_IP
```

2. Create .env.production :

```
bash
```

```
cd /home/ec2-user/authichain/app
nano .env.production
```

3. Add All Variables:

```
```bash
Database
DATABASE_URL="postgresql://authichain_admin:password@authichain-
db.xxx.rds.amazonaws.com:5432/authichain"

NextAuth
NEXTAUTH_SECRET="your-secret"
NEXTAUTH_URL="http://YOUR_EC2_IP:3000"

Stripe
STRIPE_SECRET_KEY="sk_live_51..."
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_live_51..."
STRIPE_WEBHOOK_SECRET="whsec_..."

NFT.Storage
NFT_STORAGE_API_KEY="eyJhbGciOi..."

App
NEXT_PUBLIC_SITE_URL="http://YOUR_EC2_IP:3000"
NODE_ENV="production"

All 15 Stripe Price IDs
(Add from STRIPE_PRICE_SETUP.md)
```
```

1. **Save and exit:** `Ctrl+X`, `Y`, `Enter`

Step 7: Run Database Migrations

1. Still on EC2:

```
bash
cd /home/ec2-user/authichain/app
npx prisma generate
npx prisma db push
```

2. Restart Application:

```
bash
pm2 restart authichain
```

3. Check Logs:

```
bash
pm2 logs authichain
```

Step 8: Create Application Load Balancer

1. Create Target Group:

```
bash
```

```
aws elbv2 create-target-group \
  --name authichain-targets \
  --protocol HTTP \
  --port 3000 \
  --vpc-id vpc-xxxxxxx \
  --health-check-path /api/health \
  --health-check-interval-seconds 30
```

2. Register EC2 Instance:

```
bash
aws elbv2 register-targets \
  --target-group-arn arn:aws:elasticloadbalancing:us-east-1:123456789:targetgroup/
authichain-targets/xxx \
  --targets Id=i-xxxxxxx
```

3. Create Load Balancer:

```
bash
aws elbv2 create-load-balancer \
  --name authichain-alb \
  --subnets subnet-xxxxxxx subnet-yyyyyyy \
  --security-groups sg-xxxxxxx
```

4. Create Listener:

```
bash
aws elbv2 create-listener \
  --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:123456789:loadbalancer/app/
authichain-alb/xxx \
  --protocol HTTP \
  --port 80 \
  --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-
east-1:123456789:targetgroup/authichain-targets/xxx
```

Step 9: Configure SSL with ACM

1. Request SSL Certificate:

```
bash
aws acm request-certificate \
  --domain-name authichain.com \
  --subject-alternative-names www.authichain.com \
  --validation-method DNS
```

2. Get Validation Records:

```
bash
aws acm describe-certificate \
  --certificate-arn arn:aws:acm:us-east-1:123456789:certificate/xxx
```

3. Add DNS Records (in Route 53 or your DNS provider):

- Copy the CNAME name and value from previous command
- Add to your DNS provider

4. Wait for Validation:

```
bash
aws acm wait certificate-validated \
  --certificate-arn arn:aws:acm:us-east-1:123456789:certificate/xxx
```

5. Add HTTPS Listener to ALB:

```
bash
aws elbv2 create-listener \
  --load-balancer-arn arn:aws:elasticloadbalancing:us-east-1:123456789:loadbalancer/app/authichain-alb/xxx \
  --protocol HTTPS \
  --port 443 \
  --certificates CertificateArn=arn:aws:acm:us-east-1:123456789:certificate/xxx \
  --default-actions Type=forward,TargetGroupArn=arn:aws:elasticloadbalancing:us-east-1:123456789:targetgroup/authichain-targets/xxx
```

Step 10: Configure Route 53 (DNS)**1. Create Hosted Zone:**

```
bash
aws route53 create-hosted-zone \
  --name authichain.com \
  --caller-reference $(date +%s)
```

2. Get ALB DNS Name:

```
bash
aws elbv2 describe-load-balancers \
  --names authichain-alb \
  --query 'LoadBalancers[0].DNSName' \
  --output text
```

3. Create A Record (Alias):

```
bash
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1234567890ABC \
  --change-batch '{
    "Changes": [{
      "Action": "CREATE",
      "ResourceRecordSet": {
        "Name": "authichain.com",
        "Type": "A",
        "AliasTarget": {
          "HostedZoneId": "Z35SXD0TRQ7X7K",
          "DNSName": "authichain-alb-xxx.us-east-1.elb.amazonaws.com",
          "EvaluateTargetHealth": false
        }
      }
    }]
  }'
```

Step 11: Set Up S3 for Static Assets (Optional)

1. Create S3 Bucket:

```
bash
aws s3 mb s3://authichain-assets
```

2. Enable Public Access:

```
bash
aws s3api put-bucket-policy \
  --bucket authichain-assets \
  --policy '{
    "Version": "2012-10-17",
    "Statement": [{
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::authichain-assets/*"
    }]
  }'
```

3. Configure Next.js to use S3:

```
javascript
// next.config.js
module.exports = {
  assetPrefix: process.env.NODE_ENV === 'production'
    ? 'https://authichain-assets.s3.amazonaws.com'
    : ''
};
```

Step 12: Set Up CloudFront (CDN)

1. Create CloudFront Distribution:

```
bash
aws cloudfront create-distribution \
  --origin-domain-name authichain-alb-xxx.us-east-1.elb.amazonaws.com \
  --default-root-object index.html
```

2. Get CloudFront Domain:

```
bash
aws cloudfront list-distributions \
  --query 'DistributionList.Items[0].DomainName' \
  --output text
```

3. Update Route 53 to point to CloudFront (instead of ALB)

Advanced Configuration

Auto Scaling

1. Create Launch Template:

```
bash
aws ec2 create-launch-template \
  --launch-template-name authichain-template \
  --version-description "AuthiChain v1" \
  --launch-template-data '{
    "ImageId": "ami-0c55b159cbfafa1f0",
    "InstanceType": "t3.medium",
    "KeyName": "authichain-key",
    "SecurityGroupIds": ["sg-xxxxxxx"],
    "UserData": "BASE64_ENCODED_SETUP_SCRIPT"
  }'
```

2. Create Auto Scaling Group:

```
bash
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name authichain-asg \
  --launch-template LaunchTemplateName=authichain-template,Version=1 \
  --min-size 2 \
  --max-size 10 \
  --desired-capacity 2 \
  --target-group-arns arn:aws:elasticloadbalancing:us-east-1:123456789:targetgroup/authichain-targets/xxx \
  --availability-zones us-east-1a us-east-1b
```

3. Create Scaling Policies:

```
```bash
Scale up when CPU > 70%
aws autoscaling put-scaling-policy \
 --auto-scaling-group-name authichain-asg \
 --policy-name scale-up \
 --scaling-adjustment 1 \
 --adjustment-type ChangeInCapacity

Scale down when CPU < 30%
aws autoscaling put-scaling-policy \
 --auto-scaling-group-name authichain-asg \
 --policy-name scale-down \
 --scaling-adjustment -1 \
 --adjustment-type ChangeInCapacity
```
```

Monitoring with CloudWatch

1. Create Dashboard:

```
bash
```

```
aws cloudwatch put-dashboard \
  --dashboard-name AuthiChain \
  --dashboard-body file://dashboard.json
```

2. Set Up Alarms:

```
bash
# High CPU alarm
aws cloudwatch put-metric-alarm \
  --alarm-name authichain-high-cpu \
  --alarm-description "Alert when CPU exceeds 80%" \
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average \
  --period 300 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2
```

Database Backups

1. Enable Automated Backups:

```
bash
aws rds modify-db-instance \
  --db-instance-identifier authichain-db \
  --backup-retention-period 30 \
  --preferred-backup-window "03:00-04:00" \
  --apply-immediately
```

2. Create Manual Snapshot:

```
bash
aws rds create-db-snapshot \
  --db-snapshot-identifier authichain-backup-$(date +%Y%m%d) \
  --db-instance-identifier authichain-db
```

Testing

Test Application

```
curl -I https://authichain.com
```

Test Database Connection

```
psql "postgresql://authichain_admin:password@authichain-db.xxx.rds.amazonaws.com:5432/authichain"
```

Load Testing

```
# Install Apache Bench
sudo yum install httpd-tools

# Test
ab -n 10000 -c 100 https://authichain.com/
```

Cost Optimization

Estimated Monthly Costs

| Service | Instance Type | Estimated Cost |
|-------------------|---------------|---------------------|
| EC2 (t3.medium) | 2 instances | \$60 |
| RDS (db.t3.micro) | Single-AZ | \$15 |
| ALB | Standard | \$25 |
| CloudFront | 1TB transfer | \$85 |
| Route 53 | Hosted zone | \$0.50 |
| Total | | ~\$185/month |

Cost Saving Tips

1. **Use Reserved Instances** (save 30-50%)
2. **Right-size instances** (monitor and adjust)
3. **Use S3 Intelligent-Tiering**
4. **Enable CloudFront caching**
5. **Clean up unused resources**

Additional Resources

- [AWS Well-Architected Framework](https://aws.amazon.com/architecture/well-architected/) (https://aws.amazon.com/architecture/well-architected/)
- [AWS Cost Explorer](https://aws.amazon.com/aws-cost-management/aws-cost-explorer/) (https://aws.amazon.com/aws-cost-management/aws-cost-explorer/)
- [AWS CLI Documentation](https://docs.aws.amazon.com/cli/) (https://docs.aws.amazon.com/cli/)

Deployment Checklist

- ☐ AWS CLI installed and configured
- ☐ RDS PostgreSQL database created
- ☐ EC2 instances launched

- [] Security groups configured
 - [] Environment variables set
 - [] Database migrations run
 - [] Application Load Balancer created
 - [] SSL certificate issued
 - [] Route 53 DNS configured
 - [] Auto Scaling configured (optional)
 - [] CloudWatch monitoring enabled
 - [] Database backups enabled
 - [] Test all functionality
 - [] Complete [PRODUCTION_CHECKLIST.md](#) (./PRODUCTION_CHECKLIST.md)
-

Congratulations! AuthiChain is now live on AWS! 🌥️🎉